

Autenticación, autorización y sesiones en APIs

Objetivos de aprendizaje

Modelar estados de identidad, distinguir autenticación de autorización, verificar rotación, expiración y revocación, y diseñar controles cruzados entre sesiones, objetos y acciones.

Prerrequisitos

HTTP, cookies, bearer tokens, estados, controles negativos y MFA.

Fundamentos técnicos

Autenticar establece una identidad; la sesión relaciona solicitudes con ese estado; autorizar decide si esa identidad puede realizar una acción sobre un objeto. Un token válido puede estar correctamente autenticado y aun así acceder a recursos indebidos por un fallo de autorización.

Modelo mental

```
anónimo -> credencial primaria -> preautenticado -> MFA -> autenticado
      -> elevación/acción sensible -> logout/expiración/revocación
      |
      +--> autorización por identidad + acción + objeto
```

Superficie de ataque

Login, refresh, logout, recuperación, cambio de contraseña, “recordar dispositivo”, endpoints de objetos y cambios de rol.

Cómo identificarla

- La sesión rota o no rota tras cambiar privilegio.
- Token viejo sigue o deja de funcionar tras refresh/logout.
- Mismo objeto responde distinto para dueño, otro usuario y anónimo.
- Timeouts se aplican en servidor, no solo en interfaz.

Preguntas que debo hacerme

1. ¿Qué material autentica cada petición?
2. ¿Qué estado representa y dónde se conserva?
3. ¿Cuándo se rota o revoca?
4. ¿Qué autorización se aplica por objeto y acción?
5. ¿Hay caché o gateway que explique la respuesta?

Prueba mínima

Construir una matriz con sesión A, sesión B y anónimo sobre recurso A y B, usando lectura y una acción no destructiva. Después probar token anterior/nuevo en refresh y logout.

Construcción progresiva del payload

En esta técnica el “payload” suele ser una secuencia: obtener sesión, capturar identificador, cambiar una sola precondition, repetir exactamente el recurso y comparar estado server-side. No se manipula un token opaco al azar.

Anatomía de los payloads

- Contexto de entrada: Authorization: Bearer <token> y path /orders/1042.

- Sintaxis original: token identifica sesión; path identifica objeto.
- Entrada controlada: token y objeto solicitado.
- Transformaciones conocidas: gateway valida firma; API resuelve sesión.
- Parser final: middleware de autenticación y política de autorización.
- Sink: lectura de pedido.
- Primitiva: acceder con sesión B a objeto A.
- Payload mínimo: petición válida de B cambiando solo 1043 por 1042.
- Significado de cada componente: el token conserva la identidad B; el path selecciona el objeto atribuido a A.
- Resultado esperado: 403 o respuesta indistinguible sin datos en una implementación segura.
- Control negativo: B->B, A->A y anónimo->A.
- Restricción observada: ambos usuarios reciben 200 para 1042.
- Por qué falla la variante básica: quitar token solo prueba autenticación.
- Hipótesis de adaptación: mantener identidad válida y cambiar objeto.
- Payload adaptado: matriz identidad-objeto.
- Por qué debería funcionar: separa autenticación de pertenencia.
- Evidencia: B recibe datos exclusivos de A reproduciblemente.
- Cuándo no funcionaría: autorización correcta o recurso público compartido.

Variaciones según el contexto

Sesión server-side, token autosuficiente y refresh token tienen ciclos distintos. Expiración idle, absoluta y de credencial no son equivalentes. Logout efectivo requiere invalidación server-side cuando el mecanismo lo permite; borrar una cookie local no basta para demostrar revocación.

Filtros y bypasses

No trates 401/403 como oráculos perfectos: gateways y aplicaciones los usan de forma inconsistente. Mantén cuerpo, método, objeto y sesión controlados; registra redirecciones y caché.

Evidencias de confirmación

Acceso indebido a objeto/acción, aceptación de una sesión que debería estar revocada o ausencia de rotación con impacto demostrable. Una cookie sin HttpOnly es una debilidad distinta, no bypass por sí sola.

Escalado de impacto

Ampliar de lectura a acciones solo si es necesario y seguro; probar cambio de contraseña, elevación o revocación con cuentas del laboratorio. Evitar tocar datos de otros usuarios reales.

Errores frecuentes

- Probar sin token y concluir autorización segura.
- Confundir endpoint público con bypass.
- No fijar caché o sesión.
- Asumir que expiración del JWT revoca refresh tokens.
- Tratar ocultación del botón como control server-side.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
sin token y con token responden igual	recurso público o token ignorado	/me y recurso privado conocido
logout borra cookie, token aún funciona	revocación solo cliente	reutilizar token desde cliente separado
token viejo y nuevo funcionan	ventana de rotación o fallo	medir duración y evento de revocación
B lee objeto A	autorización por objeto ausente	matriz B->B/A y acción distinta
200 con cuerpo vacío	autorización en lógica o caché	comparar datos, cabeceras y backend
sesión cambia tras login	rotación esperada	comprobar que la anterior se invalida

Mitigaciones

Estado server-side coherente, rotación tras cambios de privilegio, expiración idle/absoluta, revocación, cookies seguras, autorización por petición y objeto, y pruebas automatizadas de matrices de acceso.

Relación con pentesting y certificaciones

Se evalúa el modelo de estado y la matriz de controles, no la cantidad de tokens manipulados.

Caso guiado

Caso A — Básico: pertenencia de pedidos

Cuenta B cambia /orders/1043 por /orders/1042 y recibe los datos de A.

Observación: B está autenticada y lee A. Qué sé: autenticación funciona; pertenencia no. Hipótesis: autorización ausente o caché. Experimento: marcadores anti-caché, varios objetos y acción de lectura. Resultado: patrón sigue el ID, no la caché. Conclusión: bypass de autorización por objeto. Siguiendo paso: demostrar alcance mínimo.

Caso de adaptación

Caso B — Logout aparente

La UI vuelve al login tras logout, pero una copia del bearer token sigue accediendo a /api/me.

Observación: interfaz cerrada, token válido. Hipótesis: token no revocable hasta expirar o endpoint de logout incompleto. Experimento: cliente separado, token de acceso y refresh por separado, tiempos. Resultado: access token sigue 15 min y refresh queda revocado. Conclusión: comportamiento documentado del diseño, no necesariamente bypass; el riesgo depende del requisito de revocación inmediata. Siguiendo paso: comparar con especificación del sistema.

Caso C — Transferencia: cambio de correo

Una acción sensible acepta la sesión autenticada hace horas sin reautenticación. No se nombra el fallo en el título.

Resolución: sesión válida -> acción de alto riesgo -> ausencia de verificación reciente. El experimento compara sesión recién autenticada y antigua, no manipula el correo. Se documenta como política de reautenticación débil si el requisito la exige.

Caso D — Falso positivo: endpoint de catálogo

/api/plans responde igual sin token y con token. El alumno sospecha token ignorado.

Experimento: /api/me y /api/billing con las mismas condiciones. Resultado: catálogo es público; privados devuelven 401. Conclusión: la igualdad no demuestra bypass.

Ejercicios

1. Construye una matriz de dos usuarios, dos objetos y dos acciones.
2. Distingue expiración de access token y revocación de refresh token.
3. Diseña un control para caché en una prueba de autorización.
4. Explica cuándo 401 frente a 403 no cambia la conclusión.

Resumen

Autenticación, sesión y autorización son capas distintas. Las pruebas útiles cambian una identidad, objeto, acción o estado cada vez y verifican el ciclo completo.

Chuleta operativa

1. Estado e identidad.
2. Token/cookie y ciclo.
3. Matriz identidad-objeto-acción.
4. Rotación y token anterior.
5. Idle/absoluta/revocación.
6. Caché y controles.

Referencias

- [OWASP Authentication Cheat Sheet](#)
- [OWASP Session Management Cheat Sheet](#)
- [PortSwigger: access control](#)
- [Concepto interno](#) (auth-session-security)

Navegación

Anterior: [MFA/OTP](#). Índice: [curso](#). Práctica: [cuaderno](#).